

OOT_测试驱动开发

一、TDD / Design for Test 核心知识

1. TDD 是什么

TDD，即 Test-Driven Development，测试驱动开发。

核心流程：

Test → Code → Refactor

也就是：

1. 先写测试用例；
2. 再写刚好能通过测试的代码；
3. 最后在测试保护下重构代码。

重点：TDD 不只是“自动测试技术”，更是一种**设计技术**。

因为先写测试会迫使程序员先思考类的接口、输入、输出和行为细节。

2. Design for Testing 的核心

要测试一个组件，必须做到：

能够控制它的输入和内部状态，并且能够观察它的输出。

所以如果代码中存在以下情况，就很难自动化测试：

- 随机数不可控；
 - 当前时间不可控；
 - 数据库、文件、网络等外部资源不可控；
 - 真实发送邮件、短信、支付请求，有副作用；
 - 直接读取控制台输入，需要人工参与；
 - 直接输出到控制台或 GUI，难以断言；
 - 方法内部直接 `new` 具体依赖，测试时无法替换；
 - 全局变量、静态状态、单例导致测试之间互相影响。
-

3. 常见修改手段

高频修改方式：

接口和实现分离

依赖注入 Dependency Injection

Mock Object / Stub / Fake

提取业务逻辑

返回结果对象，而不是直接打印

把时间、随机数、数据库、邮件等外部依赖抽象成接口

考试答题模板：

这段代码难以自动化测试，因为它在方法内部直接依赖了不可控或不可观察的因素，例如随机数、系统时间、数据库、文件、网络、控制台输入输出、GUI 或具体对象创建。这样测试无法控制输入或内部状态，也无法稳定观察输出，导致测试不可重复、不稳定或需要人工参与。

修改方法是将核心业务逻辑与外部依赖分离，对外部依赖定义接口，并通过依赖注入传入。测试时使用 Mock / Stub / Fake 对象替代真实依赖，使输入可控、输出可观察，从而实现自动化单元测试。

二、例题 1：掷骰子游戏中的随机数不可控

小题背景

系统中有一个掷骰子游戏。玩家掷出 6 点获胜，否则失败。现在代码直接在 `play()` 方法中创建随机数。

Bad Java Code

```
import java.util.Random;

class DiceGame {
    public int play() {
        int point = new Random().nextInt(6) + 1;

        if (point == 6) {
            System.out.println("You win");
            return 100;
        } else {
            System.out.println("You lose");
            return 0;
        }
    }
}
```

无法自动化测试的部分

1. `new Random().nextInt(6) + 1`：随机数在方法内部直接生成，测试无法控制骰子点数。
2. `System.out.println(...)`：方法直接输出到控制台，测试不方便观察和断言输出。
3. 随机数生成、游戏规则、显示输出混在一个方法里，职责不清晰。

为什么不好

自动化测试需要结果稳定可重复。

如果随机数不可控，那么同一个测试有时通过、有时失败，无法稳定验证 `point == 6` 和 `point != 6` 两种情况。

重构方案

引入 `Dice` 接口，把掷骰子的行为从 `DiceGame` 中分离出去。

运行时使用 `RandomDice`，测试时使用 `FakeDice` 返回固定点数。

Refactored Java Sketch

```
interface Dice {
    int roll();
}

class RandomDice implements Dice {
    public int roll() {
        return new java.util.Random().nextInt(6) + 1;
    }
}

class DiceGame {
    private Dice dice;

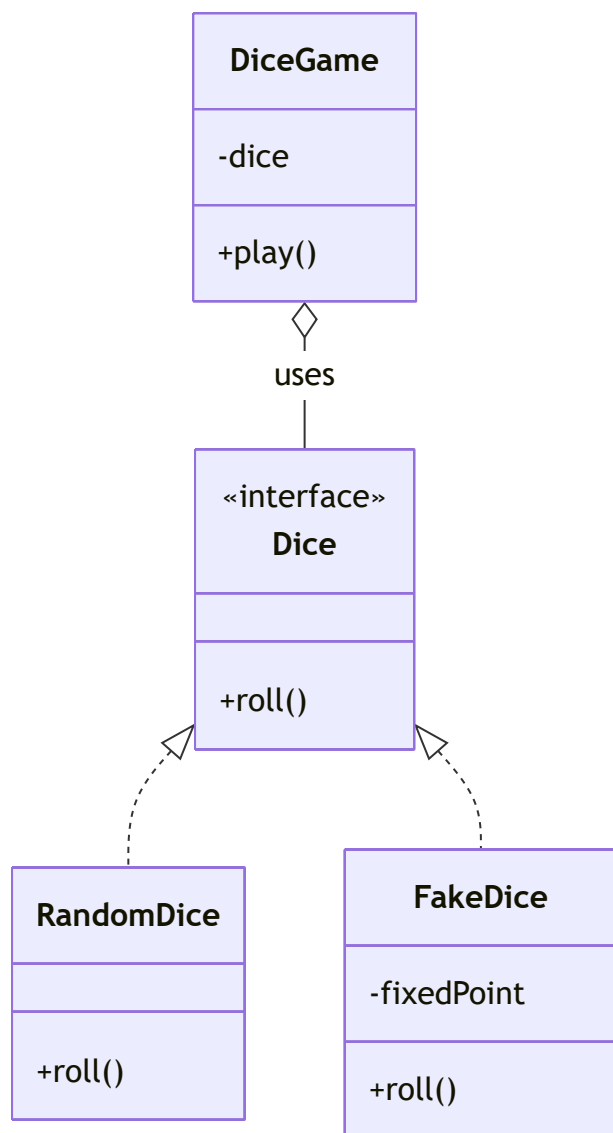
    public DiceGame(Dice dice) {
        this.dice = dice;
    }

    public int play() {
        int point = dice.roll();

        if (point == 6) {
            return 100;
        } else {
            return 0;
        }
    }
}
```

```
class FakeDice implements Dice {  
    public int roll() {  
        return 6;  
    }  
}
```

重构后 Mermaid UML



测试思路

```
DiceGame game = new DiceGame(new FakeDice());  
assertEquals(100, game.play());
```

答题模板

该代码难以自动化测试，因为 `DiceGame` 在 `play()` 方法内部直接创建随机数，测试无法控制骰子点数，导致测试结果不稳定。同时，方法直接输出到控制台，测试不方便断言输出。可以将骰子抽象为 `Dice` 接口，并通过依赖注入传入。测试时使用 `FakeDice` 返回固定点数，从而使输入可控、结果可观察。

三、例题 2：系统当前时间不可控

小题背景

电商系统在周日给用户打 8 折。当前代码直接使用系统当前日期判断是否周日。

Bad Java Code

```
import java.time.LocalDate;
import java.time.DayOfWeek;

class DiscountService {
    public double calculatePrice(double price) {
        if (LocalDate.now().getDayOfWeek() == DayOfWeek.SUNDAY) {
            return price * 0.8;
        }
        return price;
    }
}
```

无法自动化测试的部分

`LocalDate.now()` 直接读取系统当前日期，测试无法控制“今天是星期几”。

为什么不好

测试结果依赖运行日期。

如果今天是周日，测试会走打折分支；如果不是周日，就不会打折。这样测试不可重复。

重构方案

引入 `DateProvider` 接口。

运行时用真实日期，测试时用固定日期。

Refactored Java Sketch

```
import java.time.LocalDate;
import java.time.DayOfWeek;

interface DateProvider {
    LocalDate today();
}

class SystemDateProvider implements DateProvider {
    public LocalDate today() {
```

```
        return LocalDate.now();
    }
}

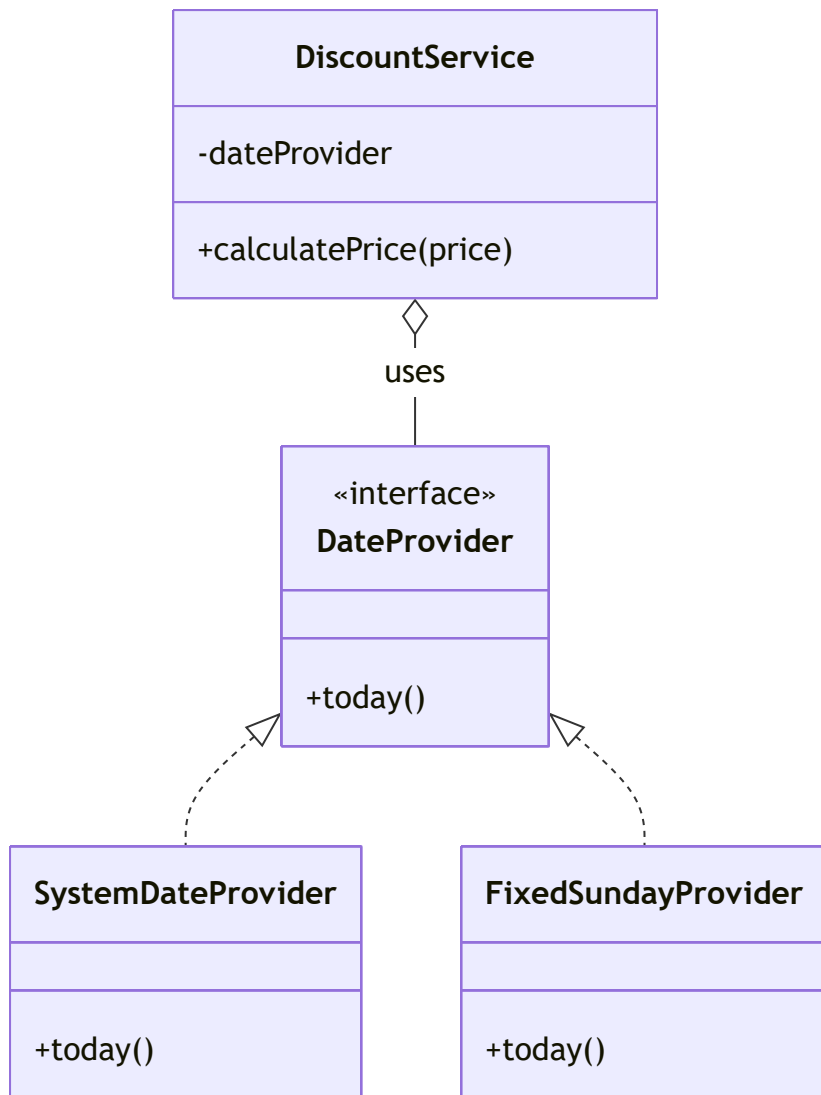
class FixedSundayProvider implements DateProvider {
    public LocalDate today() {
        return LocalDate.of(2026, 6, 14); // Sunday
    }
}

class DiscountService {
    private DateProvider dateProvider;

    public DiscountService(DateProvider dateProvider) {
        this.dateProvider = dateProvider;
    }

    public double calculatePrice(double price) {
        if (dateProvider.today().getDayOfWeek() == DayOfWeek.SUNDAY) {
            return price * 0.8;
        }
        return price;
    }
}
```

重构后 Mermaid UML



测试思路

```
DiscountService service = new DiscountService(new FixedSundayProvider());
assertEquals(80.0, service.calculatePrice(100.0));
```

答题模板

该代码难以自动化测试，因为它直接调用 `LocalDate.now()` 获取系统当前时间，测试无法控制时间状态，导致测试结果依赖运行日期。可以将时间来源抽象为 `DateProvider` 接口，并通过依赖注入传入。测试时使用固定日期的 `Provider`，从而稳定测试周日折扣逻辑。

四、例题 3：银行转账依赖数据库，使用 Mock Object

小题背景

银行系统中，`AccountService` 负责从一个账户转账到另一个账户。它需要通过 `AccountManager` 查找账户并更新账户。真实的 `AccountManager` 可能连接数据库，因此单元测试不应该依赖真实数据库。

Java Code after Design for Test

```
interface AccountManager {  
    Account findAccountForUser(String userId);  
    void updateAccount(Account account);  
}
```

```
class Account {  
    private String id;  
    private long balance;  
  
    public Account(String id, long balance) {  
        this.id = id;  
        this.balance = balance;  
    }  
  
    public void debit(long amount) {  
        balance -= amount;  
    }  
  
    public void credit(long amount) {  
        balance += amount;  
    }  
  
    public long getBalance() {  
        return balance;  
    }  
}
```

```
class AccountService {  
    private AccountManager accountManager;  
  
    public void setAccountManager(AccountManager manager) {  
        this.accountManager = manager;  
    }  
  
    public void transfer(String senderId, String beneficiaryId, long amount) {  
        Account sender = accountManager.findAccountForUser(senderId);  
        Account beneficiary = accountManager.findAccountForUser(beneficiaryId);  
  
        sender.debit(amount);  
        beneficiary.credit(amount);  
  
        accountManager.updateAccount(sender);  
        accountManager.updateAccount(beneficiary);  
    }  
}
```



```
}  
}
```

Mock Object

```
import java.util.HashMap;  
import java.util.Map;  
  
class MockAccountManager implements AccountManager {  
    private Map<String, Account> accounts = new HashMap<>();  
  
    public void addAccount(String userId, Account account) {  
        accounts.put(userId, account);  
    }  
  
    public Account findAccountForUser(String userId) {  
        return accounts.get(userId);  
    }  
  
    public void updateAccount(Account account) {  
        // do nothing in test  
    }  
}
```

测试用例示意

```
MockAccountManager mock = new MockAccountManager();  
  
Account sender = new Account("1", 200);  
Account beneficiary = new Account("2", 100);  
  
mock.addAccount("1", sender);  
mock.addAccount("2", beneficiary);  
  
AccountService service = new AccountService();  
service.setAccountManager(mock);  
  
service.transfer("1", "2", 50);  
  
assertEquals(150, sender.getBalance());  
assertEquals(150, beneficiary.getBalance());
```

无法自动化测试的部分

如果没有接口和 Mock，真实 `AccountManager` 可能访问数据库。数据库是外部资源，不适合作为单元测试的必要条件。

为什么不好

单元测试应该快速、稳定、独立。

依赖真实数据库会让测试慢、不稳定，还需要准备数据库状态。测试失败时也难以判断是业务逻辑错了，还是数据库环境错了。

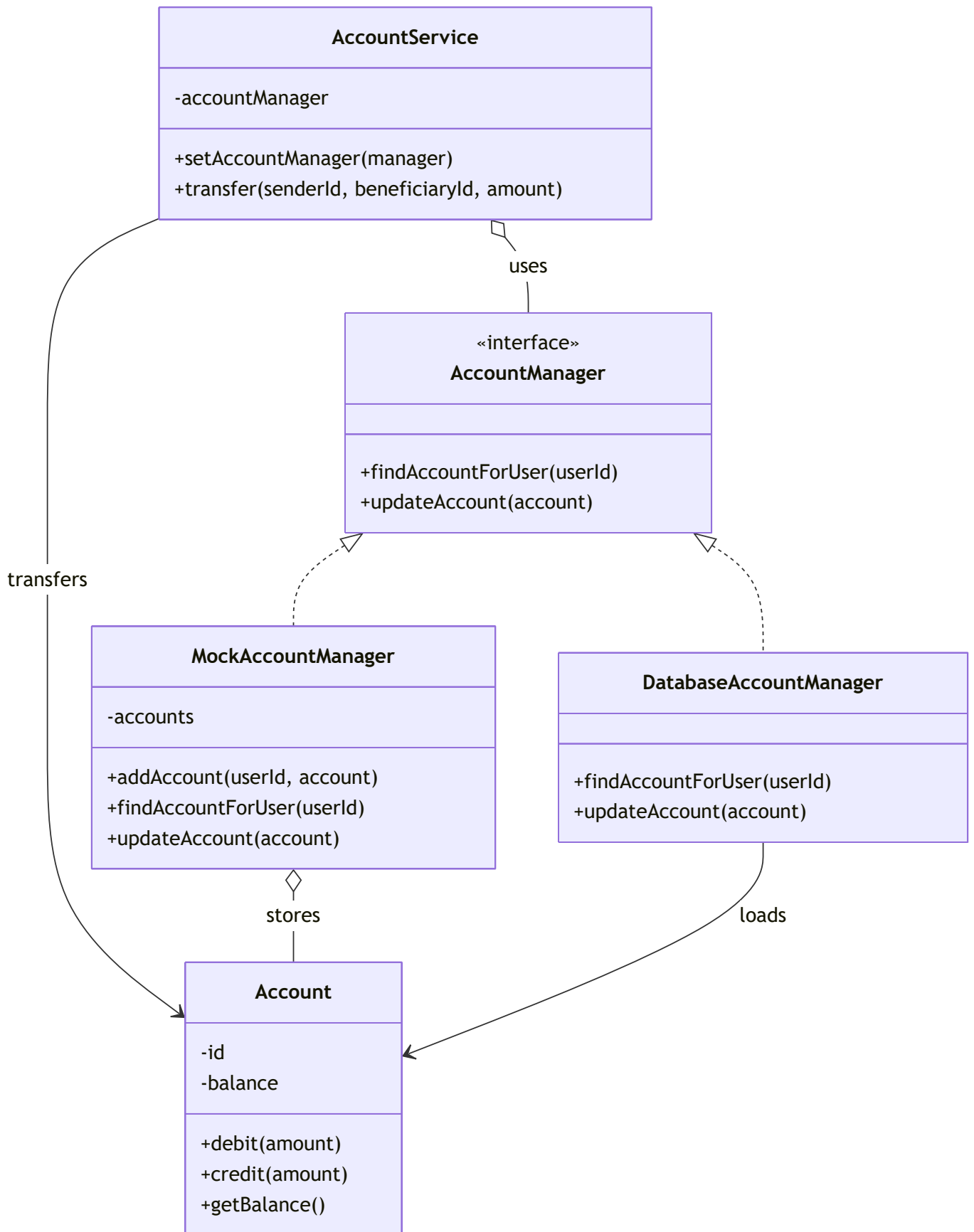
重构方案

将 `AccountManager` 抽象为接口。

`AccountService` 依赖接口而不是具体数据库实现。

测试时注入 `MockAccountManager`，用内存中的 `Map` 模拟账户数据。

重构后 Mermaid UML



答题模板

该代码的核心业务是测试 `AccountService.transfer()`，但真实 `AccountManager` 可能访问数据库，导致单元测试依赖外部环境。应将 `AccountManager` 设计为接口，并让 `AccountService` 通过 setter 或构造函数接收该依赖。测试时使用 `MockAccountManager` 替代真实数据库实现，使测试只关注转账逻辑。这样输入可控、输出可观察，测试快速且可重复。

五、例题 4：直接发送邮件，有外部副作用

小题背景

订单系统创建订单后，会给用户发送邮件。当前代码在 `OrderService` 内部直接创建 `EmailSender` 并发送真实邮件。

Bad Java Code

```
class EmailSender {
    public void send(String email, String message) {
        // connect to SMTP server and send real email
    }
}

class OrderService {
    public void createOrder(String email) {
        // create order in system

        EmailSender sender = new EmailSender();
        sender.send(email, "Order created");
    }
}
```

无法自动化测试的部分

1. `new EmailSender()`： `OrderService` 内部直接创建具体依赖，测试无法替换。
2. `sender.send(...)`：可能真的发送邮件，有外部副作用，不适合单元测试。

为什么不好

单元测试不应该真的发送邮件。

真实邮件服务可能慢、不可用、需要网络配置，也可能产生真实副作用。

重构方案

引入 `MessageSender` 接口，通过依赖注入传入。

运行时用 `EmailSender`，测试时用 `MockSender`。

Refactored Java Sketch

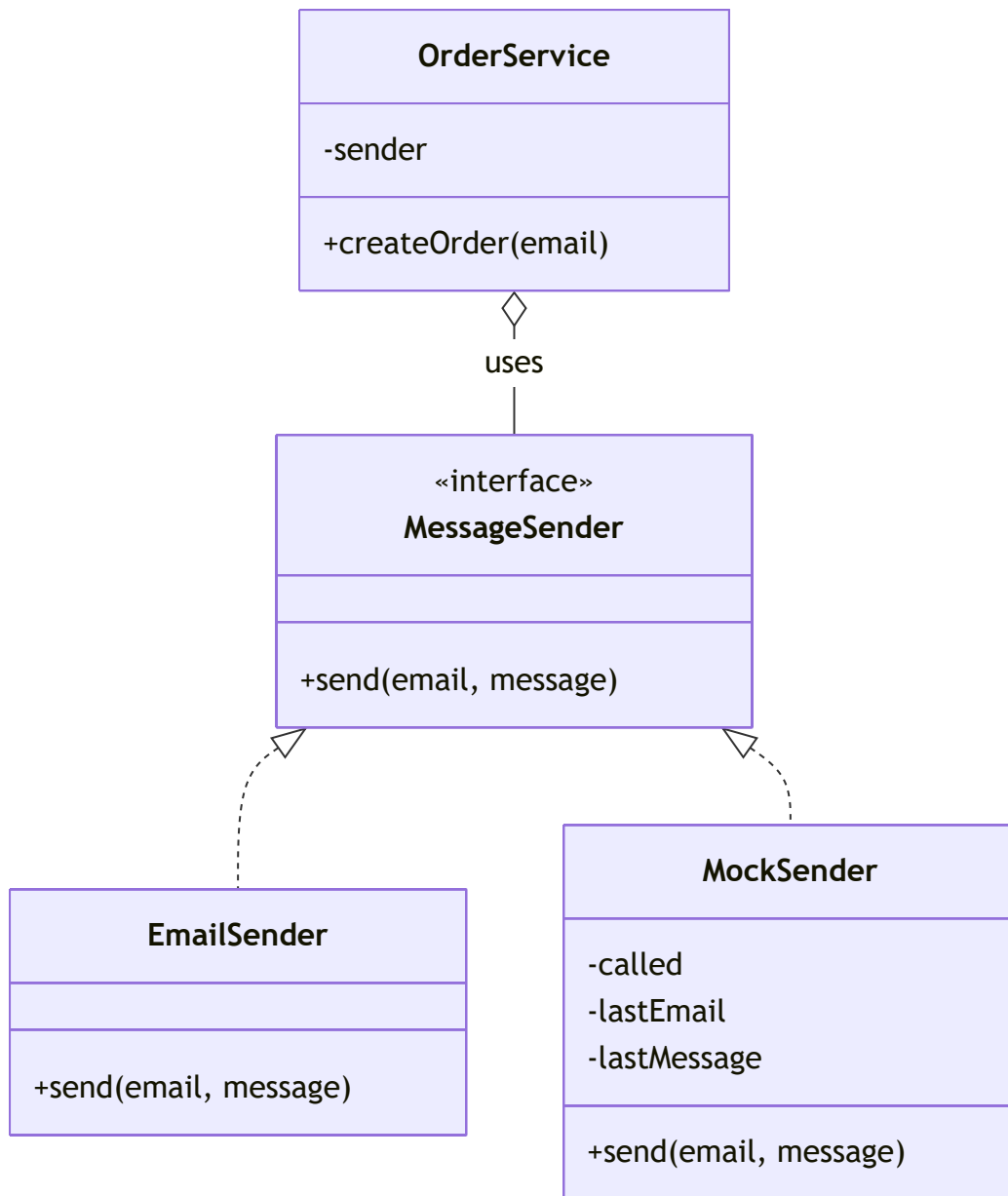
```
interface MessageSender {
    void send(String email, String message);
}
```

```
class EmailSender implements MessageSender {  
    public void send(String email, String message) {  
        // send real email  
    }  
}
```

```
class MockSender implements MessageSender {  
    public boolean called = false;  
    public String lastEmail;  
    public String lastMessage;  
  
    public void send(String email, String message) {  
        called = true;  
        lastEmail = email;  
        lastMessage = message;  
    }  
}
```

```
class OrderService {  
    private MessageSender sender;  
  
    public OrderService(MessageSender sender) {  
        this.sender = sender;  
    }  
  
    public void createOrder(String email) {  
        // create order in system  
        sender.send(email, "Order created");  
    }  
}
```

重构后 Mermaid UML



测试思路

```
MockSender mock = new MockSender();
OrderService service = new OrderService(mock);

service.createOrder("a@test.com");

assertEquals(true, mock.called);
assertEquals("a@test.com", mock.lastEmail);
assertEquals("Order created", mock.lastMessage);
```

答题模板

该代码难以自动化测试，因为 `OrderService` 在内部直接创建 `EmailSender`，导致依赖不可替换；同时真实发送邮件具有外部副作用。可以定义 `MessageSender` 接口，并通过构造函数注入。测试时注入 `MockSender`，验证 `send()` 是否被调用以及参数是否正确，而不需要真的发送邮件。

六、例题 5：业务逻辑和控制台输入输出混在一起

小题背景

用户注册程序直接从控制台读取用户名，并直接打印注册结果。现在希望为注册逻辑编写自动化单元测试。

Bad Java Code

```
import java.util.Scanner;

class RegisterProgram {
    public void run() {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Input username: ");
        String username = scanner.nextLine();

        if (username.length() >= 3) {
            System.out.println("register success");
        } else {
            System.out.println("username too short");
        }
    }
}
```

无法自动化测试的部分

1. `Scanner(System.in)`：需要人工输入，自动化测试无法稳定控制。
2. `System.out.println(...)`：输出到控制台，测试不方便断言。
3. 注册规则和界面交互混在一起。

为什么不好

自动化测试不能依赖人工输入，也不应该主要通过观察控制台文字判断业务逻辑是否正确。核心业务逻辑应该独立于 UI。

重构方案

提取 `RegisterService`，让业务逻辑接收参数并返回结果。
控制台输入输出留在 UI 层。

Refactored Java Sketch

```
class RegisterResult {
    private boolean success;
    private String message;

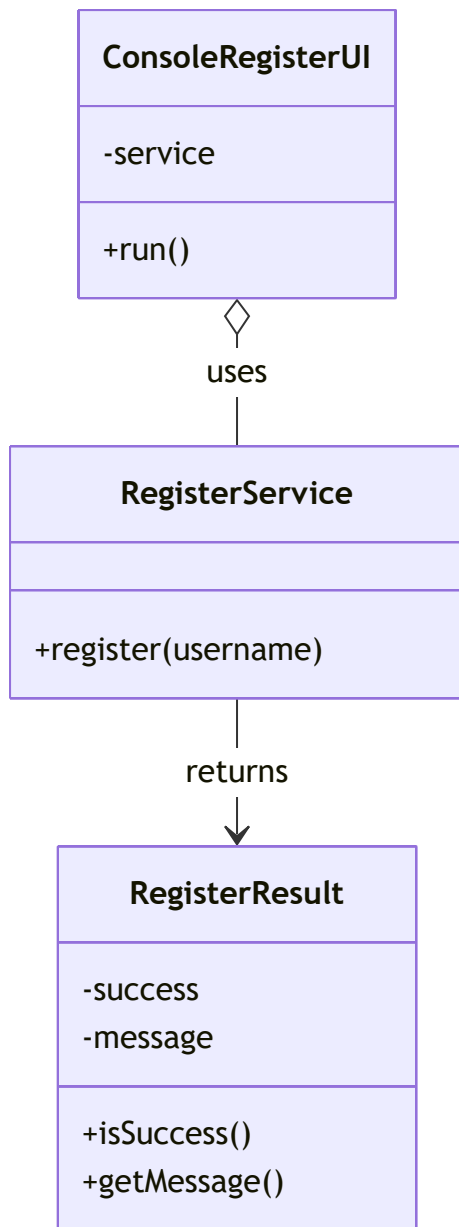
    public RegisterResult(boolean success, String message) {
        this.success = success;
        this.message = message;
    }

    public boolean isSuccess() {
        return success;
    }

    public String getMessage() {
        return message;
    }
}

class RegisterService {
    public RegisterResult register(String username) {
        if (username.length() >= 3) {
            return new RegisterResult(true, "register success");
        } else {
            return new RegisterResult(false, "username too short");
        }
    }
}
```

重构后 Mermaid UML



测试思路

```
RegisterService service = new RegisterService();

RegisterResult result = service.register("tom");

assertEquals(true, result.isSuccess());
assertEquals("register success", result.getMessage());
```

答题模板

该代码难以自动化测试，因为它直接从 `System.in` 读取输入，需要人工参与；同时直接输出到控制台，测试难以观察和断言结果。可以将业务逻辑从控制台 UI 中提取出来，让 `RegisterService.register()` 接收用户名参数并返回 `RegisterResult`。这样测试可以直接传入输入并断言返回结果。

七、例题 6：全局配置和静态状态导致隐藏依赖

小题背景

支付系统使用全局静态变量保存支付服务器地址。`PaymentService` 直接读取这个全局变量。

Bad Java Code

```
class GlobalConfig {
    public static String paymentUrl = "https://pay.example.com";
}

class PaymentService {
    public boolean pay(double amount) {
        String url = GlobalConfig.paymentUrl;
        // connect to url and pay
        return true;
    }
}
```

无法自动化测试的部分

1. `GlobalConfig.paymentUrl` 是全局静态状态。
2. `PaymentService` 隐式依赖这个全局配置。
3. 不同测试可能修改同一个全局变量，互相影响。

为什么不好

测试之间应该相互独立。

全局变量会让测试依赖共享状态，导致一个测试的修改影响另一个测试。

重构方案

将配置封装为对象，并注入到 `PaymentService` 中。

或者定义 `PaymentGateway` 接口，把真实支付请求隔离掉。

Refactored Java Sketch

```
class PaymentConfig {
    private String paymentUrl;

    public PaymentConfig(String paymentUrl) {
        this.paymentUrl = paymentUrl;
    }

    public String getPaymentUrl() {
        return paymentUrl;
    }
}
```

```

    }
}

interface PaymentGateway {
    boolean pay(double amount);
}

class RealPaymentGateway implements PaymentGateway {
    private PaymentConfig config;

    public RealPaymentGateway(PaymentConfig config) {
        this.config = config;
    }

    public boolean pay(double amount) {
        String url = config.getPaymentUrl();
        // connect to real payment server by url
        return true;
    }
}

class MockPaymentGateway implements PaymentGateway {
    public boolean pay(double amount) {
        return true;
    }
}

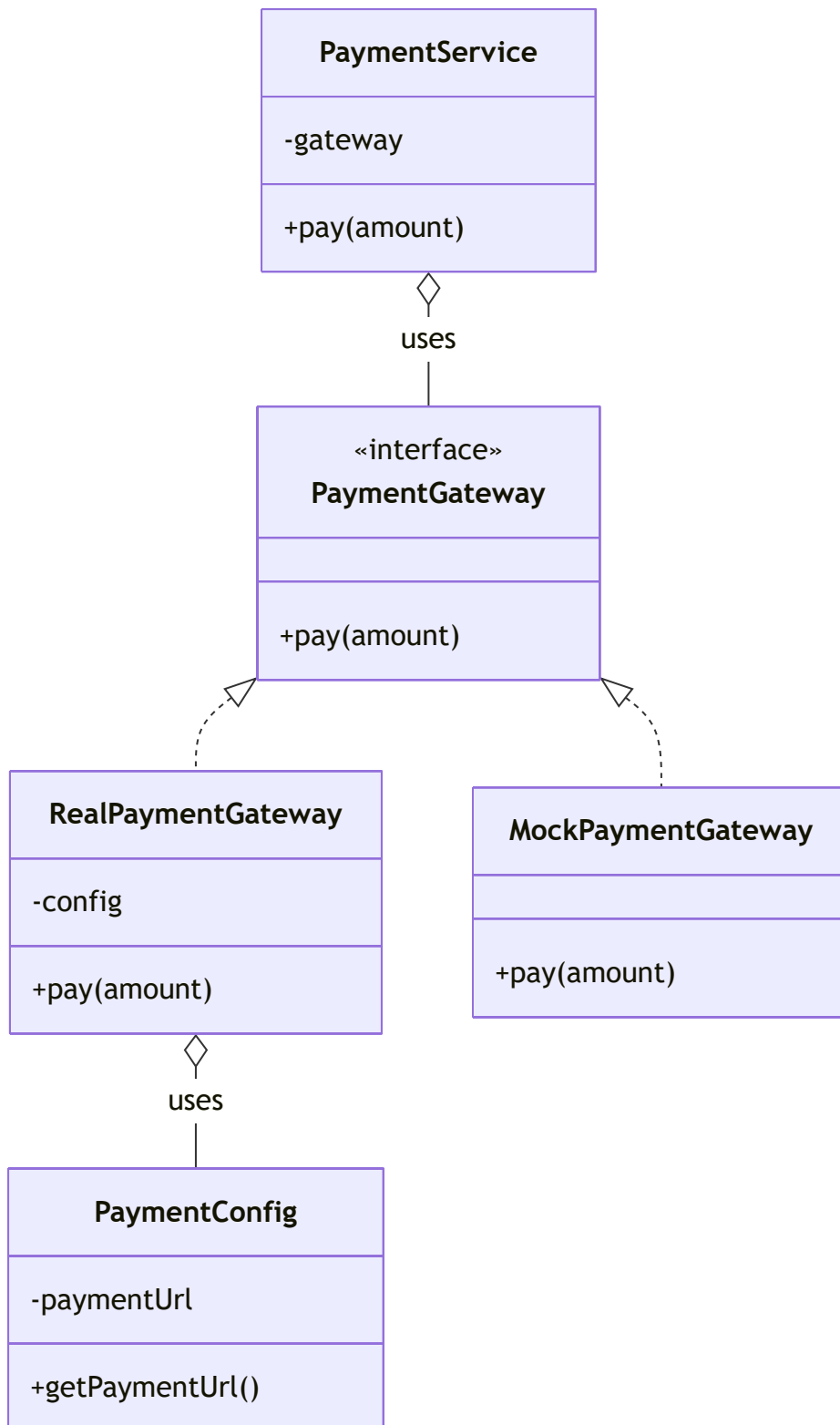
class PaymentService {
    private PaymentGateway gateway;

    public PaymentService(PaymentGateway gateway) {
        this.gateway = gateway;
    }

    public boolean pay(double amount) {
        return gateway.pay(amount);
    }
}

```

重构后 Mermaid UML



测试思路

```
PaymentConfig config = new PaymentConfig("https://pay.example.com");
PaymentGateway gateway = new RealPaymentGateway(config);
PaymentService service = new PaymentService(gateway);
PaymentGateway mockGateway = new MockPaymentGateway();
PaymentService service = new PaymentService(mockGateway);

assertEquals(true, service.pay(100.0));
```

答题模板

该代码难以自动化测试，因为它依赖全局静态配置，形成隐藏依赖，测试之间可能互相影响。可以将配置封装为对象，并将外部支付请求抽象为 `PaymentGateway` 接口。`PaymentService` 依赖接口并通过构造函数注入，测试时使用 `MockPaymentGateway`，避免真实外部请求和全局共享状态。

八、例题 7：代码缺陷修复前先写测试

小题背景

购物车系统中发现一个 bug：当购物车为空时，`averagePrice()` 会除以 0。现在要按照 TDD 思路修复这个 bug。

Bad Java Code

```
class ShoppingCart {
    private List<Item> items;

    public double averagePrice() {
        double sum = 0;
        for (Item item : items) {
            sum += item.getPrice();
        }
        return sum / items.size();
    }
}
```

问题

当 `items.size() == 0` 时，会发生除以 0 的问题。

TDD 修复步骤

Step 1: 先写一个能暴露 bug 的测试

```
public void testAveragePriceForEmptyCart() {
    ShoppingCart cart = new ShoppingCart();

    assertEquals(0.0, cart.averagePrice());
}
```

第一次运行时，这个测试应该失败。

Step 2: 写最少代码让测试通过

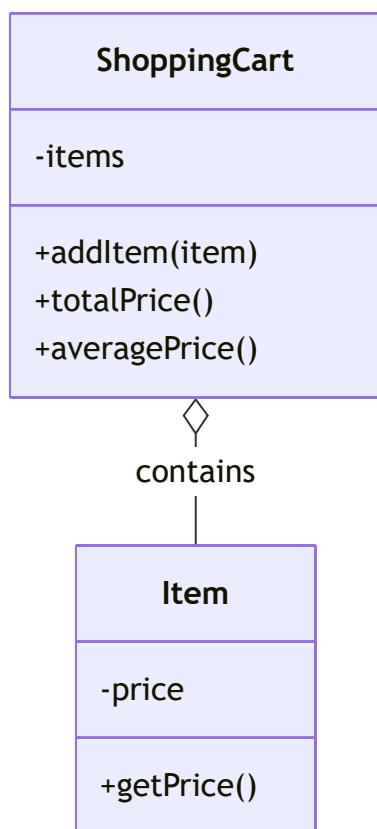
```
public double averagePrice() {
    if (items.size() == 0) {
        return 0.0;
    }

    double sum = 0;
    for (Item item : items) {
        sum += item.getPrice();
    }
    return sum / items.size();
}
```

Step 3: 在测试保护下重构

如果有必要，可以把求和逻辑提取为 `totalPrice()`。

Mermaid UML



答题模板

按照 TDD，修复 bug 时应先写一个能够暴露该 bug 的测试用例，并确保该测试一开始失败。然后编写最少代码使测试通过，最后在测试保护下重构。对于空购物车平均价格的问题，应先写

`testAveragePriceForEmptyCart()`，再在 `averagePrice()` 中处理空集合情况。

九、期末高频判断表

代码现象	为什么难以自动化测试	修改方法
<code>new Random()</code>	输入不可控，测试不稳定	注入 <code>RandomProvider</code> / <code>Dice</code>
<code>LocalDate.now()</code>	时间不可控，测试依赖日期	注入 <code>DateProvider</code> / <code>Clock</code>
直接连数据库	依赖外部环境，慢且不稳定	Repository / Manager 接口 + Mock
直接读写文件	有环境依赖和副作用	FileService 接口 + Mock
直接发邮件/短信	真实副作用，不适合单元测试	MessageSender 接口 + Mock
直接发起支付/网络请求	外部系统不可控	Gateway 接口 + Mock
方法内部 <code>new</code> 具体依赖	依赖不可替换	Dependency Injection
<code>System.in</code>	需要人工输入	输入作为参数 / <code>InputReader</code> 接口
<code>System.out.println</code>	输出难断言	返回 <code>Result</code> 对象
GUI 弹窗	难以自动判断	UI 与业务逻辑分离
全局变量 / 静态状态	隐藏依赖，测试互相影响	注入配置对象
<code>Thread.sleep()</code>	测试慢且不稳定	分离调度和业务逻辑
只修改私有状态	输出不可观察	提供查询方法或返回值

十、简答题模板

1. 什么是 TDD?

TDD 是测试驱动开发，基本流程是 Test → Code → Refactor。它要求先写单元测试，再写实现代码，由测试来决定需要什么类、接口和方法。TDD 不只是自动测试技术，更是一种设计技术，因为它能帮助程序员澄清接口和行为细节。

2. 为什么 TDD 有助于设计?

因为先写测试会迫使开发者从使用者角度思考对象的接口、输入、输出和行为。为了让测试容易编写，代码通常会被设计成低耦合、高内聚、依赖抽象、易替换依赖的结构。

3. 什么是 Design for Testing?

Design for Testing 指为了让软件容易测试而进行设计。核心是：测试一个组件时，必须能够控制它的输入和内部状态，并且能够观察它的输出。常用方法包括接口与实现分离、依赖注入、Mock Object、分离业务逻辑和外部资源。

4. 什么是 Mock Object?

Mock Object 是测试中用来替代真实协作对象的假对象。被测代码可以调用 Mock，Mock 会按照测试预设返回结果或记录调用情况。它适合替代数据库、外部系统、邮件服务、支付接口等不可控或有副作用的真实依赖。

5. 好的单元测试应该满足什么要求？

好的单元测试应该快速、独立、可重复、无副作用，并且能够自动判断成功或失败。测试之间不应该互相依赖，测试运行前后系统状态应该保持一致。